

CCIS

كلية علوم الحاسب والمعلومات
COLLEGE OF COMPUTER &
INFORMATION SCIENCES



Flutter

جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY



IS487 Emerging Topics in IS

Dr. Abbas Malik
amaalik@psu.edu.sa

SQLite Db in Flutter Apps

- Understand SQLite and why it is used
- Learn how to integrate SQLite into a mobile app
- Explore CRUD operations
- Build a data model and database helper
- Apply SQLite to real app features

What is SQLite

- Lightweight, file-based relational database
- No server required
- Fast and reliable for offline apps
- Perfect for mobile apps - persistent storage
- Cross Platform

Why need a local DB?

- Offline functionality
- Fast data access
- Structured and relational Data
- Persistent storage
- Cross-platform

Why need a local DB?

- Sensitive local-only Data
- Sync-Later Workflow
- Large Local Datasets
- Remote or Unstable Environment

Packages in Flutter

sqflite

- Support transactions and batches
- Automatic version management during open
- Helpers for insert/update/delete queries
- DB operation in background thread
- Supports: Android, iOS, MacOS

drift

- Flexible
- Feature rich
- Modular
- Safe
- Fast
- Cross-platform
- Tested and production ready

sqlite vs drift

Aspect	sqlite	drift	Winner
Learning Curve	Very low	Medium to high	sqlite
Type Safety	None	Excellent	drift
Boilerplate	High	Medium to low	drift
Runtime safety	Easy to make typos	Almost impossible	drift
Complex Queries/Joins	Manual SQL	Fluent Dart API	drift
Performance	Very good	Almost same	Tie
Reactive/Streams	Manual	Build-in	drift
Schema Migration	Manual	Very good build-in	drift
Latest Support	Complicated limited	Excellent	drift
Web/Desktop Support	Limited Experimental	Good native support	drift
Maintenance & Risk	Stable	Stable	Tie

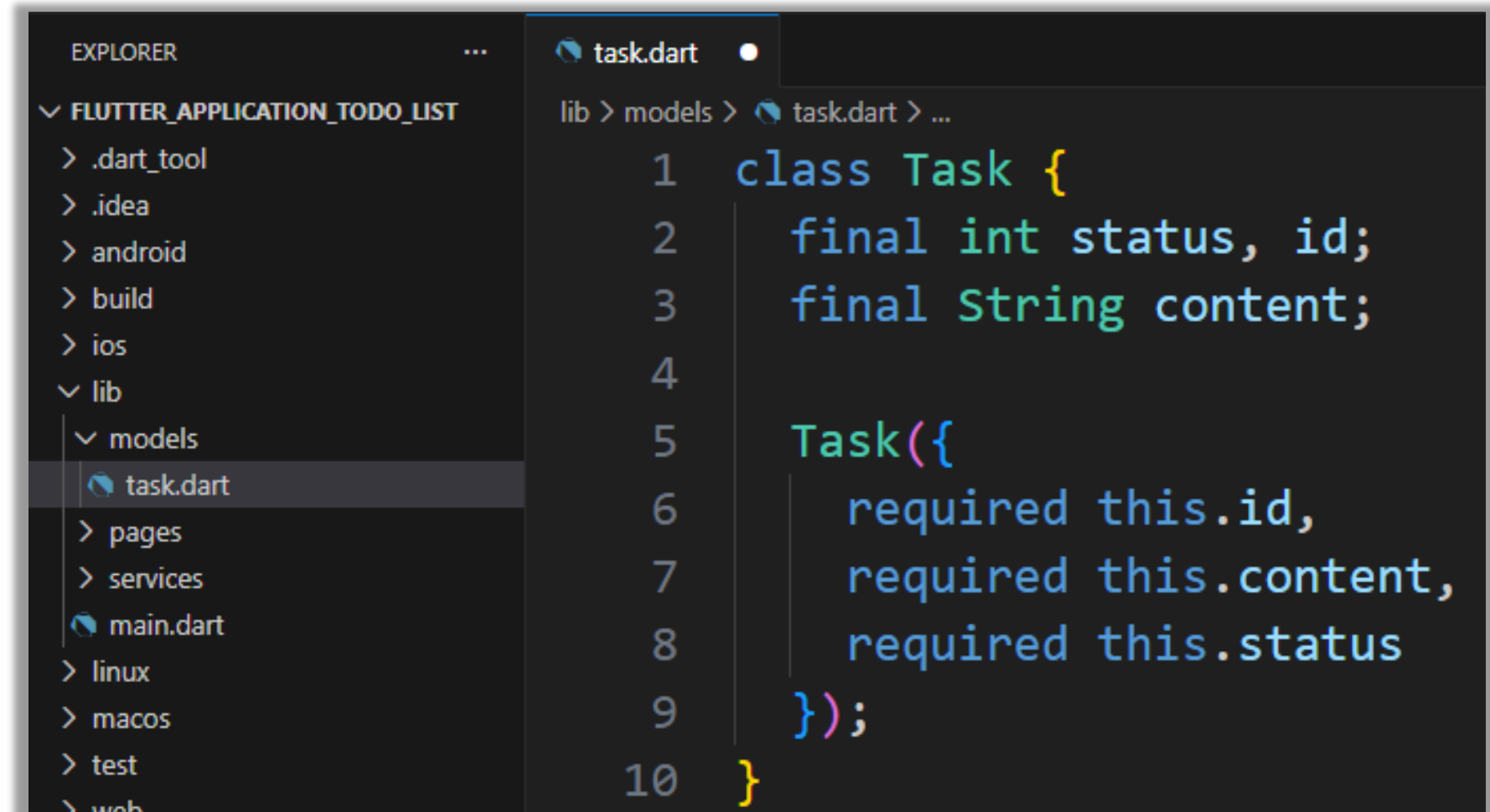
ToDo App

- Tasks - table
- Packages: **sqflite** and **path**

Tasks	
PK	<u>id</u>
	content
	status

ToDo App - Task Model

- Task class



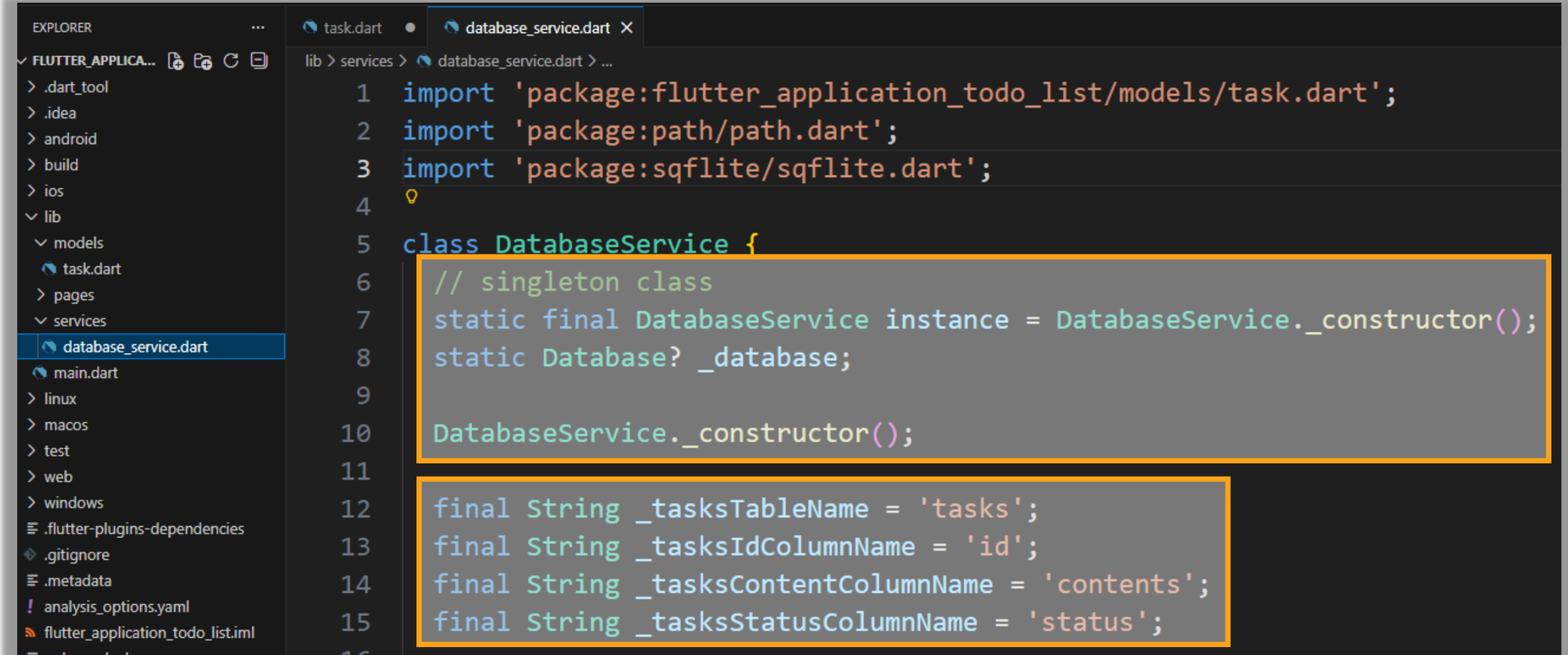
The screenshot shows an IDE with the Explorer panel on the left and the task.dart file open in the editor. The Explorer panel shows the project structure with the following folders and files:

- FLUTTER_APPLICATION_TODO_LIST
 - .dart_tool
 - .idea
 - android
 - build
 - ios
 - lib
 - models
 - task.dart (selected)
 - pages
 - services
 - main.dart
 - linux
 - macos
 - test
 - web

The task.dart file contains the following Dart code:

```
1 class Task {  
2     final int status, id;  
3     final String content;  
4  
5     Task(  
6         required this.id,  
7         required this.content,  
8         required this.status  
9     );  
10 }
```

ToDo App - DatabaseService class



The screenshot shows an IDE with the Explorer panel on the left and the editor on the right. The Explorer panel shows the project structure for 'FLUTTER_APPLICATION_TODO_LIST'. The editor displays the 'database_service.dart' file, which contains the following code:

```
1 import 'package:flutter_application_todo_list/models/task.dart';
2 import 'package:path/path.dart';
3 import 'package:sqflite/sqflite.dart';
4
5 class DatabaseService {
6     // singleton class
7     static final DatabaseService instance = DatabaseService._constructor();
8     static Database? _database;
9
10    DatabaseService._constructor();
11
12    final String _tasksTableName = 'tasks';
13    final String _tasksIdColumnName = 'id';
14    final String _tasksContentColumnName = 'contents';
15    final String _tasksStatusColumnName = 'status';
```

ToDo App - DatabaseService class

```
Future<Database> getDatabase() async {  
    final databaseDirPath = await getDatabasesPath();  
    final databasePath = join(databaseDirPath, 'tasks_db.db');  
    final database = await openDatabase(  
        databasePath,  
        version: 1,  
        onCreate: (db, version) {  
            db.execute('''  
                CREATE TABLE $_tasksTableName (  
                    $_tasksIdColumnName INTEGER PRIMARY KEY,  
                    $_tasksContentColumnName TEXT NOT NULL,  
                    $_tasksStatusColumnName INTEGER NOT NULL  
                )  
            ''');  
        },  
    );  
    return database;  
}
```

Tasks	
PK	<u>id</u>
	content
	status

ToDo App - DatabaseService class

```
Future<Database> get database async {  
    if (_database != null) return _database!;  
    _database = await getDatabase();  
    return _database!;  
}
```

ToDo App - DatabaseService class

Create

```
// insert record
void addTasks(String content) async {
    final db = await database;
    await db.insert(_tasksTableName, {
        _tasksContentColumnName: content,
        _tasksStatusColumnName: 0,
    });
}
```

ToDo App - DatabaseService class

Read

```
// read record
Future<List<Task>> getTasks() async {
  final db = await database;
  final data = await db.query(_tasksTableName);
  List<Task> tasks = data
    .map(
      (element) => Task(
        id: element['id'] as int,
        content: element['contents'] as String,
        status: element['status'] as int,
      ),
    )
    .toList();
  return tasks;
}
```

ToDo App - DatabaseService class

Update

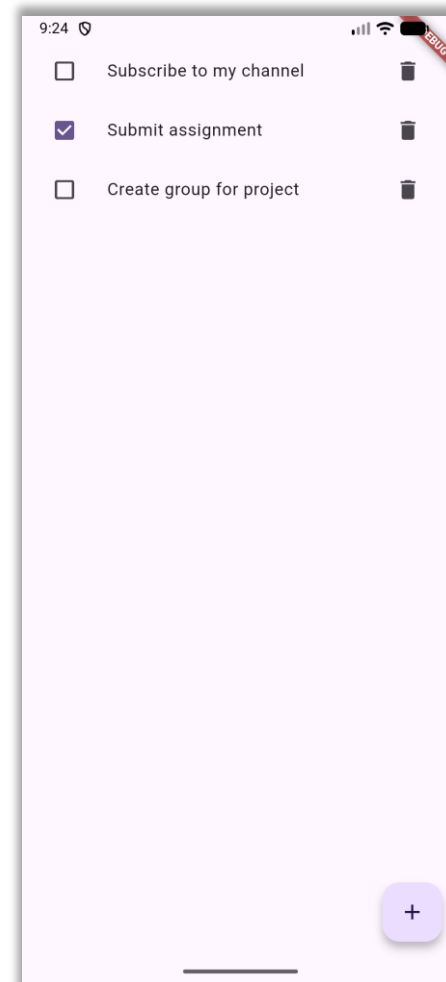
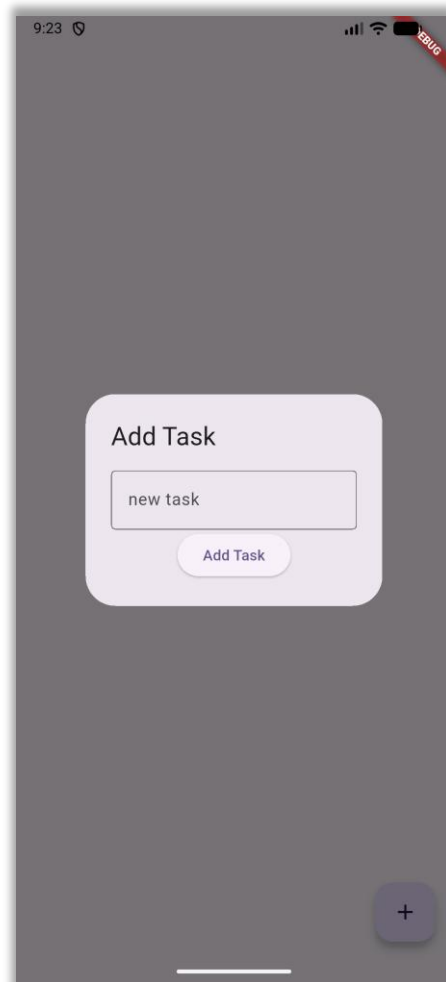
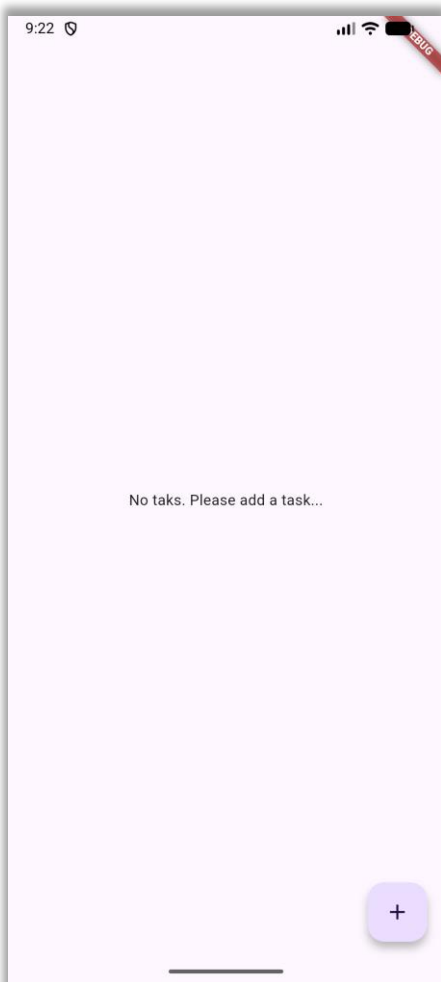
```
void updateTaskStatus(int id, int status) async {  
    final db = await database;  
    await db.update(  
        _tasksTableName,  
        {_tasksStatusColumnName: status},  
        where: 'id = ?',  
        whereArgs: [id],  
    );  
}
```

ToDo App - DatabaseService class

Delete

```
void deleteTask(int id) async {  
    final db = await database;  
    await db.delete(_tasksTableName, where: 'id = ?', whereArgs: [id]);  
}
```

ToDo App - HomePage



Drift Package

- **Type Safety:** no SQL parsing the list
- **Stream Queries:** lets widget watch for changes
- **Fluent Queries:** methods and classes to write queries and automatically get their results
- **Type-safe SQL:** can write SQL, built in parser and analyser
- **Migration utils**

Column Types

Dart Type	Drift Column	SQL Type ¹
<code>int</code>	<code>late final age = integer>()()</code>	<code>INTEGER</code>
<code>BigInt</code> (as 64-bit, see why)	<code>late final age = int64>()()</code>	<code>INTEGER</code>
<code>String</code>	<code>late final name = text>()()</code>	<code>TEXT</code>
<code>bool</code>	<code>late final isAdmin = boolean>()()</code>	<code>INTEGER (1 or 0)</code>
<code>double</code>	<code>late final height = real>()()</code>	<code>REAL</code>
<code>Uint8List</code>	<code>late final image = blob>()()</code>	<code>BLOB</code>
<code>DriftAny</code>	<code>late final value = sqliteAny>()()</code>	<code>ANY (for STRICT tables)</code>
<code>DateTime</code> (see options)	<code>late final createdAt = dateTime>()()</code>	<code>INTEGER</code> or <code>TEXT</code>
Your own	See type converter docs .	Depending on type
Enums	intEnum or textEnum .	<code>INTEGER</code> or <code>TEXT</code>
Postgres Types	See postgres docs .	Depending on type

Defining Table

```
class Task extends Table {  
  IntColumn get id => integer().autoIncrement();  
  TextColumn get content => text().withLength(max: 250);  
  BoolColumn get status => boolean();  
}
```

Custom Primary Key

```
class Profiles extends Table {  
    late final email = text();  
  
    @override  
    Set<Column<Object>> get primaryKey => {email};  
}
```

Foreign Key

```
class Albums extends Table {  
    late final id = integer().autoIncrement();  
    late final name = text();  
    late final artist = integer().references(Artists, #id);  
}
```

```
class Artists extends Table {  
    late final id = integer().autoIncrement();  
    late final name = text();  
}
```

Defining Table

- Nullable Column
- Default values
- References
- Validation checks
- Unique
- Index

https://drift.simonbinder.eu/dart_api/tables/

ToDo App

- Tasks - table
- Packages: **drift**, **drift_flutter** and **path_provider**
- Dev Packages: **build_runner** and **drift_dev**

Documentation:

<https://drift.simonbinder.eu/>

Tasks	
PK	<u>id</u>
	content
	status

ToDo App - Task Model

- Task class

```
class Task extends Table {  
  IntColumn get id => integer().autoIncrement();  
  TextColumn get content => text().withLength(max: 250);  
  BoolColumn get status => boolean();  
}
```

ToDo App - Database Service

```
part 'database_service.g.dart';

@DriftDatabase(tables: [Task])
class DatabaseService extends _$DatabaseService {
  DatabaseService() : super(_openConnection());

  @override
  int get schemaVersion => 1;
}
```

ToDo App - Database Service

```
LazyDatabase _openConnection() {  
  return LazyDatabase(() async {  
    return driftDatabase(  
      name: 'todo_app_db.db',  
      web: DriftWebOptions(  
        sqlite3Wasm: Uri.parse('sqlite3.wasm'),  
        driftWorker: Uri.parse('drift_worker.dart.js'),  
      ),  
    );  
  });  
});  
}
```

dart run build_runner build

ToDo App - Database Service

```
1 // GENERATED CODE - DO NOT MODIFY BY HAND
2
3 part of 'database_service.dart';
4
5 // ignore_for_file: type=lint
6 class $TaskTable extends Task with TableInfo<$TaskTable, TaskData> {
7   @override
8   final GeneratedDatabase attachedDatabase;
9   final String? _alias;
10   $TaskTable(this.attachedDatabase, [this._alias]);
11   static const VerificationMeta _idMeta = const VerificationMeta('id');
12   @override
13   late final GeneratedColumn<int> id = GeneratedColumn<int>(
14     'id',
15     aliasedName,
16     false,
17     hasAutoIncrement: true,
18     type: DriftSqlType.int,
19     requiredDuringInsert: false,
20     defaultConstraints: GeneratedColumn.constraintTsAlways(
```

ToDo App - Database Service

- Create

```
Future<int> insertTask(TaskCompanion taskObj) {  
    return into(task).insert(taskObj);  
}
```

ToDo App - Database Service

- Read

```
Future<List<TaskData>> getAllTasks() {  
    return select(task).get();  
}
```

ToDo App - Database Service

- Read selected with where

```
Future<List<TaskData>> getSelectedTasks() {  
  return (  
    select(task)..where(  
      (t) => t.id.isSmallerOrEqualValue(5)  
    )  
  ).get();  
}
```

https://drift.simonbinder.eu/dart_api/select/

ToDo App - Database Service

- Update

```
Future<bool> updateTask(TaskData taskData) {  
    return update(task).replace(taskData);  
}
```

ToDo App - Database Service

- Delete

```
Future<void> deleteTask(int id) {  
    return (  
        delete(task)..where((t) => t.id.equals(id))  
    ).go();  
}
```

References

- <https://www.sqlite.org/index.html>
- <https://pub.dev/packages/sqlite>
- <https://drift.simonbinder.eu>